NOVA CASA · TELEMETRÍA IOT EN SALESFORCE

Documentación Técnica

Sprint 2 · Juan Diego Velásquez · John Alejandro Pastor · Octubre 2026

Introducción

En este documento se describe la solución final, luego de las etapas de Discovery y desarrollo. Aquí se explican los distintos elementos que componen la organización, con sus detalles y justificaciones.

El código completo vive en el repositorio; este documento resume lo importante: las definiciones clave, cómo se activa la solución, cómo funciona y cómo se orquesta. El porqué de cada decisión queda registrado en `docs/decisiones.md`.

Datos del proyecto:

- Repositorio: `jpastorSFDC/nova-casa-telemetria`, rama `main`.
- Org de trabajo: `nova-cdo`, Enterprise trial que expira el 19 de noviembre de 2026.
- Documentos de apoyo en el repo: `README.md`, `docs/acceso.md` (seguridad), `docs/entrega.md` (contrato del mensaje y evidencia de escenarios) y `docs/decisiones.md` (decisiones y riesgos).

1. Qué hace la solución

Centraliza en Salesforce las señales de sensores IoT de los edificios de Nova Casa (temperatura, presión de agua, consumo de agua y energía, y conectividad de cámaras): las recibe del simulador del proveedor, las clasifica contra umbrales configurables, mantiene la última lectura vigente por activo y tipo de medición, abre una intervención (`Case`) ante cada crítica sin duplicarla y se la muestra al operador. El problema, las personas y lo que ve cada una están en la **Documentación Funcional, sección 1**.

Las reglas de negocio están en `docs/br-201-210.md`, numeradas de BR-201 a BR-210.

2. Arquitectura end to end

El flujo completo, desde el simulador hasta la pantalla del operador, tiene estos pasos:

1. `TelemetriaIngesta` pide páginas de señales al simulador por HTTP.

2. Por cada mensaje con identidad válida deja una fila en `Log_Senial__c` con resultado Publicada (un reenvío reutiliza la existente) y publica su evento `Senial_Sensor__e`; el mensaje sin identidad válida queda Rechazada y no se publica.
3. El trigger `SenialSensorTrigger` recibe los eventos y llama a `SenialSensorHandler`.
4. El handler clasifica cada señal contra los `Umbral__c`, elige una ganadora por activo y tipo de medición, actualiza la `Lectura_Vigente__c` y, si es crítica, abre la intervención con `IntervencionService`.
5. El log queda en su estado final (Procesada, Atrasada, Superada, Conflicto, Rechazada o Fallida).
6. El operador consulta todo en el componente `novaCasaActivosOperador`.

Qué pasos son síncronos y cuáles asíncronos, con el porqué de cada uno, está en **Arquitectura y Diseño, sección 5.7**.

> **IDEA QUE ATRAVIESA EL DISEÑO**
>
> **Publicar no es procesar.** `TelemetriaIngesta` solo recibe y publica; la clasificación y el guardado los hace el suscriptor del evento.

3. Modelo de datos

3.1 Objetos estándar reutilizados

La solución reutiliza `Account` (edificio), `Asset` (activo) y `Case` (intervención), y versiona `Contact` solo por su compartición. Por qué se usan estándar y cuáles son los campos personalizados de `Account` y `Asset` está en **Modelo de Datos, sección 2**. Estos son los campos personalizados de `Case` (intervención), con su `Origin = 'Telemetria'` en las automáticas:

Campo	Para qué
`Identidad_Senal__c`	Identidad (`source|messageId`) de la señal que abrió el caso. Único; un reenvío de esa identidad mapea al mismo caso sin tocarlo.
`Edificio__c`, `Tipo_Activo__c`, `Severidad_Activo__c`	Contexto del activo.
`Tiempo_Abierto_Horas__c`	Antigüedad del caso.
`Ultima_Critica_Medicion__c`, `Ultima_Critica_Valor__c`, `Ultima_Critica_Unidad__c`, `Ultima_Critica_At__c`	Última señal crítica del activo que refrescó el caso (sección «Última señal crítica» del layout, solo lectura).
`Ultima_Critica_Identidad__c`	Identidad de esa última crítica (técnica; la leen admin e ingesta).

3.2 Objetos personalizados

`Log_Senial__c`, la bitácora de cada señal, con una fila por identidad `source|messageId` (salvo los rechazos de ingesta sin identidad válida, que usan una clave derivada de un hash):

Campo	Para qué
`Identidad__c`	Clave única `source|messageId` (en los rechazos de ingesta sin identidad válida, una clave derivada de un hash). Evita duplicados por upsert.
`Source__c`, `Message_Id__c`	Origen y clave del mensaje.
`Resultado__c`	Estado: Publicada, Procesada, Rechazada, Atrasada, Superada, Conflicto o Fallida (el picklist incluye además Duplicada, reservada y sin uso).
`Motivo__c`	Razón del rechazo o del estado, para trazabilidad (BR-209).
`Measurement_Type__c`, `Value__c`, `Unit__c`	Medición cruda.
`Occurred_At__c`, `Publicado_At__c`, `Procesado_At__c`	Marcas de tiempo de origen, publicación y procesamiento.
`Retraso_Publicacion_Seg__c`, `Latencia_Procesamiento_Seg__c`	Métricas de latencia.
`Reenvios__c`, `Ultimo_Reenvio__c`	Conteo de reenvíos de la misma clave.
`Conflictos__c`, `Ultimo_Conflicto__c`, `Ultimo_Conflicto_Detalle__c`	Conteo de conflictos (la misma clave con otro contenido), momento del último y detalle de lo que trajo la entrega. Qué resultado conserva la fila original está en **Modelo de Datos, sección 4** (D047).
`Caso__c`	Lookup al `Case` que abrió esa señal o al que quedó enlazada (D039), si fue crítica ganadora.
`Huella__c`, `Delivery_Id__c`, `Message_Type__c`	Hash del contenido, identificador de entrega y tipo de mensaje.
`Accion_Reintento__c`	Fórmula de texto de solo lectura: según `Resultado__c`, `Reenvios__c`, `Conflictos__c` y, en una Rechazada, `Motivo__c`, dice si hace falta reintentar con «Traer señales» (la identidad evita duplicar) o qué hay que corregir antes (D014, D047, D055; BR-209).

`Lectura_Vigente__c`, la última lectura por activo y medición, con una fila por `Clave__c`:

Campo	Para qué
`Clave__c`	ExternalId del `activo|tipo de medición`. Garantiza la unicidad.
`Asset__c`, `Edificio__c`	Activo y edificio.
`Tipo_Medicion__c`, `Valor__c`, `Unidad__c`	La medición vigente.
`Severidad__c`, `Severidad_Nivel__c`	Severidad resultante: 0 Normal, 1 Advertencia, 2 Crítica.
`Occurred_At__c`, `Antiguedad_Minutos__c`	Momento de origen y antigüedad.
`Identidad_Senal__c`, `Sensor_Id__c`	Trazabilidad a la señal ganadora.

`Umbral__c`, las reglas de clasificación configurables, con una fila por regla, editable por el coordinador (BR-204):

Campo	Para qué
`Tipo_Activo__c`, `Tipo_Medicion__c`, `Unidad_Esperada__c`	A qué señales aplica.
`Operador__c`	Comparación: `LT`, `LE`, `GT` o `GE`.
`Valor__c`	Umbral contra el que se compara.
`Severidad__c`, `Severidad_Nivel__c`	Severidad si la regla se cumple: Advertencia, Critica o Invalida.
`Activo__c`, `Descripcion__c`	Marca de regla activa y descripción.

3.3 Platform Event `Senial_Sensor__e`

Un Platform Event es un mensaje que se publica y se consume en el momento, sin guardarse como registro. Es la pieza que desacopla la recepción del procesamiento: la ingesta publica el evento de cada mensaje con identidad válida y se olvida; el procesamiento ocurre aparte, en el suscriptor. Esto permite recibir volúmenes grandes sin bloquear, y que un fallo al procesar no afecte a la recepción.

Quién lo publica y quién lo consume está en las secciones 2 y 4: el trigger lo recibe en lotes de hasta 200, como el usuario de integración configurado con `PlatformEventSubscriberConfig`. El evento es efímero: no se guarda como registro (no se consulta con SOQL). El registro permanente de cada señal es el `Log_Senial__c`; el evento solo transporta los datos de la recepción al procesamiento.

Tiene 17 campos de texto, agrupados por propósito:

Grupo	Campos
Identidad y origen	`Source__c`, `Message_Id__c`, `Delivery_Id__c`, `Message_Type__c`
Ubicación del activo	`Building_Id__c`, `Asset_Id__c`, `Asset_Type__c`, `Sensor_Id__c`
Medición	`Measurement_Type__c`, `Value__c`, `Unit__c`
Tiempos	`Occurred_At__c` (origen), `Published_At__c` (publicación)
Conectividad (cámaras)	`Comm_Status__c`, `Gap_Seconds__c`, `Last_Seen_At__c`, `Episode_Id__c`

El contrato completo (formato, qué se rechaza y ejemplos válido e inválido) está en `docs/entrega.md`, sección «Message contract».

4. Clases Apex

La solución usa objetos estándar cuando alcanzan y objetos personalizados solo donde el modelo los necesita; el Apex es el que pide el brief: un suscriptor programático y lógica preparada para colecciones, para validar y procesar las señales. Las clases de la ingesta corren en modo sistema, para no depender de los permisos del operador; la pantalla corre con los permisos del usuario, con unas pocas excepciones en modo sistema que se comprueban antes en el servidor (**Arquitectura y Diseño, sección 7.3**, punto S5).

Clase	Tipo	Qué hace	Corre como
`TelemetriaIngesta`	Queueable	Pide páginas al simulador; por cada mensaje con identidad válida deja un `Log_Senial__c` Publicada (un reenvío reutiliza la fila existente) y publica `Senial_Sensor__e`. Si falla la publicación, la fila queda Fallida; el mensaje sin identidad válida queda Rechazada y no se publica. Guarda el avance de la sesión (cursor) en Platform Cache para continuar entre corridas.	Usuario que la inicia
`TelemetriaIngestaProgramada`	Schedulable	Encola `TelemetriaIngesta` cada 5 minutos con 12 cron jobs. Es la ingesta automática (US-201).	Usuario que la agenda
`SimuladorClient`	Cliente HTTP	Hace `POST /session` y `GET /telemetry` por Named Credential. No hace DML ni publica. El token y el cursor nunca se registran.	Hereda del llamador
`SesionSimulador`	Utilidad	Guarda el cursor y el resumen de la última corrida en Platform Cache, nunca en registros, porque el cursor es secreto.	Sistema
`SenialSensorHandler`	Handler del trigger	Clasifica cada evento contra `Umbral__c`, elige la ganadora por activo y medición (desempate D003), actualiza `Lectura_Vigente__c` y, si es crítica, pide la intervención. Protege cada evento para que un fallo no tumbe el lote.	Sistema, como el usuario de integración
`IntervencionService`	Servicio	Abre o enlaza la intervención (`Case`) de una señal crítica ganadora. La primera crítica de un activo abre el caso; mientras siga abierto, las siguientes críticas del mismo activo (cualquier medición) se enlazan a él y refrescan sus campos `Ultima_Critica_*` si son más recientes (US-205, D039). Un reenvío de la identidad que abrió el caso no lo toca. Bloquea el activo (`FOR UPDATE`) para que dos lotes no abran dos casos.	Sistema, solo ingesta
`OpenCaseCheck`	Utilidad	Responde sí o no a la pregunta de si un activo ya tiene un `Case` abierto, en modo sistema, para evitar duplicados aunque el usuario no vea ese `Case`.	Sistema
`UmbralValidador`	Validador	Al guardar un `Umbral__c`, impide rangos contradictorios, como que la Advertencia quede inalcanzable frente a la Crítica.	Usuario
`CatalogoService`	Carga inicial	Trae el catálogo del simulador y crea o actualiza edificios y activos sin duplicar; siembra los umbrales de referencia una sola vez.	Usuario
`ActivosOperadorController`	Controlador LWC de lectura	Alimenta el monitor: edificios, activos con severidad, historial de señales, estado del activo, resumen del edificio y qué botones ve el usuario.	Usuario (tres lecturas del log en modo sistema)
`IntervencionOperadorController`	Controlador LWC de escritura	Crea intervención manual, seguimiento (estado y comentario) y cierre en bloque. Verifica los permisos personalizados en el servidor.	Usuario, con el alta del `Case` en modo sistema
`IngestaController`	Controlador LWC del botón	Botón «Traer señales»: valida el permiso, arranca `TelemetriaIngesta` y muestra los contadores de la última corrida (D020).	Usuario

Triggers:

- `SenialSensorTrigger` sobre `Senial_Sensor__e` (after insert) llama a `SenialSensorHandler.procesar`.

- `UmbralTrigger` sobre `Umbral__c` (before insert, before update) llama a `UmbralValidador.validar`.

5. Clasificación por umbrales

Es la lógica central, en `SenialSensorHandler.clasificar`. Por cada señal:

1. Filtra los `Umbral__c` por tipo de activo y tipo de medición, y luego por unidad esperada. Si la unidad no coincide, la señal queda como `MedicionIncompatible`; si no hay umbrales para esa combinación, queda como `SinUmbral` (son estados internos; en el log la señal queda Rechazada con su motivo, D032).
2. Recorre los umbrales en orden de severidad, de Inválida a Crítica a Advertencia, y devuelve la primera que se cumple. La comparación aplica el `Operador__c` sobre el `Valor__c`. Si no se cumple ninguna, la señal es Normal.
3. Según la severidad: Inválida se marca Rechazada; Normal, Advertencia y Crítica son candidatas a actualizar la lectura vigente; Crítica, además, pasa a `IntervencionService`, que abre o enlaza la intervención del activo (**sección 4**).

Una señal atrasada (con `occurredAt` más viejo) no pisa la lectura vigente actual (BR-203). Por cada activo y medición gana una sola señal; las demás quedan como evidencia: Atrasada si son más antiguas, Superada si empataban en la hora y perdieron el desempate (D003).

5.1 Valores de referencia de los umbrales

Son los valores con que `CatalogoService.loadCatalog()` siembra `Umbral__c` la primera vez que carga cada combinación de tipo de activo y medición (recurso estático `NovaCasaUmbralesSemilla`, D054); la unidad sale del catálogo del simulador. Son los valores iniciales: el coordinador los edita en la pestaña Umbrales sin desplegar, así que los de la org pueden ser otros. Se prueban en el orden Inválida, Crítica y Advertencia, y gana la primera que se cumple; si ninguna se cumple, la señal es Normal. En el límite exacto, `>` y `<` no se cumplen (30 °C es Normal) y `≥` y `≤` sí (D019).

Tipo de activo	Medición	Unidad	Advertencia	Crítica	Inválida (se rechaza)
`TECHNICAL_ROOM`	`TEMPERATURE`	`CELSIUS`	< 18 o > 30	< 15 o > 35	≤ -11 o ≥ 81
`WATER_METER`	`WATER_CONSUMPTION`	`LITER_PER_15_MIN`	> 400	> 650	< 0 o > 5000
`ENERGY_METER`	`ENERGY_CONSUMPTION`	`KWH_PER_15_MIN`	> 40	> 65	< 0 o > 1000
`WATER_PUMP`	`WATER_PRESSURE`	`BAR`	< 2.5 o > 4	< 1.5 o > 5	> 20
`SECURITY_CAMERA`	`CAMERA_CONNECTIVITY`	`SECONDS`	> 90	> 180	-

6. Componentes LWC

Son los de la pantalla del operador:

Componente	Rol
`novaCasaActivosOperador`	Monitor principal: lista de activos, filtros por edificio, severidad y tipo, antigüedad, y estados de carga, vacío y error (BR-207).
`novaCasaEdificioResumen`	Tarjeta de resumen del edificio, con conteos por severidad.
`novaCasaActivoEstado`	Estado y detalle de un activo.
`novaCasaHistorialSenales`	Historial de señales de un activo; desde D048 no se muestra en ninguna pantalla (el componente queda en el repositorio, con Jest).
`novaCasaEtiquetas`	Etiquetas de antigüedad y estado, muestra «hora del simulador» cuando la fecha de origen viene adelantada.
`novaCasaIlustracion`	Ilustración SVG por tipo de activo (o de edificio), teñida según el nivel de severidad (HP-07, D042).
`novaCasaIntervencion`	Página de registro de la intervención (`Case`), de solo lectura: muestra el estado actual del activo con la lectura que lo fija y su antigüedad, la prioridad, la última crítica registrada, el tiempo que lleva abierta y las señales enlazadas al caso (HP-07); en una intervención manual, el motivo. La ruta de estados es el Path nativo y el estado se cambia en la pestaña Detalles (D043, D044, D046). Se monta en la flexipage `Nova_Casa_Intervencion_Record_Page`.
`novaCasaCrearIntervencion`	Ventana «Nueva intervención» para abrir una intervención manual, con el formato de un diálogo nativo de nuevo registro: muestra el activo, el edificio, el estado actual, la lectura que lo fija y la prioridad que tendrá. El servidor valida el permiso y que no haya otra abierta (HP-07, D045).

La tolerancia de reloj del simulador es la constante `FUTURE_TOLERANCE_MS`, de 5 minutos, en `novaCasaActivosOperador`. Es solo visual y no afecta la clasificación.

7. Orquestación: cómo se activa la ingesta

Hay dos formas de activarla y un mecanismo de continuidad:

- **Manual,** con el botón «Traer señales» (D020): `IngestaController.traerSenales` llama a `TelemetriaIngesta.iniciar`.
- **Automática** (US-201): `TelemetriaIngestaProgramada` encola `TelemetriaIngesta` cada 5 minutos, con 12 cron jobs en los minutos 0, 5, ..., 55. Se agenda con `scripts/apex/programar-ingesta.apex` y se quita con `scripts/apex/desprogramar-ingesta.apex`. Mientras los jobs estén agendados, para redesplegar las clases de ingesta hay que desprogramar, desplegar y volver a programar.
- **Continuidad de sesión** (HP-05, D025): el cursor se guarda en Platform Cache, así una corrida sigue donde quedó la anterior y, al releer el final de un flujo terminado, no vuelve a publicar lo ya procesado

(D040).

8. Seguridad y permisos

El modelo de acceso completo (OWD, jerarquía de roles, perfil, permission sets por persona, qué ve cada persona y qué acciones se validan en el servidor) está en **Permisos de Acceso**; la decisión es D022 y la fuente en el repositorio, `docs/acceso.md`. Lo que importa para el código:

- La ingesta y el procesamiento corren en modo sistema para no depender de los permisos del operador, y el procesamiento corre como el usuario de integración; los controladores de la pantalla son `with sharing` y consultan `WITH USER_MODE` (**Arquitectura y Diseño, sección 7**).
- **Permisos personalizados:** `Nova_Casa_Crear_Intervencion`, `Nova_Casa_Seguir_Intervencion` y `Nova_Casa_Traer_Senales`.
- Cada acción de escritura valida su permiso en el servidor. Ocultar el botón en la pantalla no es la única defensa (BR-208).

9. Pruebas

- Pruebas Apex por clase (`*Test.cls`): clasificación, volúmenes de 200 o más registros, duplicados, orden temporal y seguridad (BR-210).
- Pruebas Jest de `novaCasaActivosOperador`, `novaCasaActivoEstado`, `novaCasaEtiquetas` y `novaCasaHistorialSenales` (la suite de `novaCasaActivoEstado` no arrancó desde #103 por `Cannot find module 'lightning/modal'`; #113 añadió el stub que faltaba); los otros cuatro componentes (`novaCasaEdificioResumen`, `novaCasaIlustracion`, `novaCasaIntervencion` y `novaCasaCrearIntervencion`) no tienen Jest y se prueban a mano.
- La organización de producción reporta 0 pruebas durante el deploy, así que las pruebas se corren aparte con `sf apex run test`.
- Evidencia de escenarios en vivo en `docs/entrega.md`, sección «Scenario runs»: cuatro corridas del 27 de octubre de 2026 (`QA_200`, `DUPLICATES` y `BOUNDARIES` dos veces), cada una con un job `TelemetriaIngesta` completado sin errores y ninguna fila en estado Publicada, y el recorrido del mismo día con cinco personas, en el que pasaron todas las comprobaciones.

10. Decisiones y riesgos

Todas las decisiones técnicas, las alternativas descartadas, los trade-offs y los riesgos están en `docs/decisiones.md`, entradas D001 a D060. Las decisiones de diseño con su alternativa y su trade-off están en **Arquitectura y Diseño, sección 9**, y los riesgos operativos, en su **sección 11**. Las que más se

citan en el código y no figuran allí: **D003** (desempate por severidad y luego `messageId`), **D010** (idempotencia por identidad), **D018 y D019** (validación de rangos contradictorios y qué pasa en el límite exacto de cada operador) y **D035** (cierre en bloque de los casos acumulados).

11. Fragmentos de código clave

Extractos del código tal como está en la confirmación `d87c049` de la rama `main` del repositorio, con sus comentarios, algunos en inglés porque así están en el repositorio. Las líneas «// …» y las llaves sueltas que cierran un bloque son marcas de omisión que se añadieron: no son código del repositorio. Cada bloque indica el archivo y las líneas, con un enlace al código completo en GitHub. Son pocos a propósito: las piezas que sostienen los acuerdos del sprint.

11.1 El disparador del Platform Event

El trigger no tiene lógica: entrega el lote de eventos (hasta 200) al manejador.

Archivo `triggers/SenialSensorTrigger.trigger`, líneas 1 a 3.

```
trigger SenialSensorTrigger on Senial_Sensor__e (after insert) {
    new SenialSensorHandler().procesar(Trigger.new);
}
```

11.2 Procesamiento de un lote

El manejador consulta los umbrales una sola vez, clasifica cada señal aislando sus fallos y persiste todo el lote junto. Un fallo imprevisto no se propaga al trigger, para no descartar el lote.

Archivo `classes/SenialSensorHandler.cls`, líneas 113 a 142.

```
/**
 * @param seniales Lote entregado por SenialSensorTrigger (hasta 200 eventos).
 */
public void procesar(List<Senial_Sensor__e> seniales) {
    senialesRecibidas += seniales.size();
    System.debug(LoggingLevel.INFO, 'SenialSensorHandler recibió ' +
seniales.size() + ' señales');
    if (seniales.isEmpty()) {
        return;
    }

    try {
        Map<String, List<Umbral__c>> umbralesPorTipo = consultarUmbrales();
        List<Clasificacion> clasificaciones = new List<Clasificacion>();
        clasificacionPorMessageId = new Map<String, Clasificacion>();
```

```
        for (Integer i = 0; i < seniales.size(); i++) {
            Senial_Sensor__e senial = seniales[i];
            Clasificacion clasificacion = clasificarSeguro(senial,
umbralesPorTipo, i);
            clasificaciones.add(clasificacion);
            clasificacionPorMessageId.put(clasificacion.messageId,
clasificacion);
            System.debug(LoggingLevel.INFO, 'Señal ' + clasificacion.messageId +
' ' + clasificacion.clave + ': '
                + clasificacion.estado + ' (nivel ' + clasificacion.nivel + ')');
        }

        persistir(clasificaciones);
    } catch (Exception e) {
        // Última red: un fallo imprevisto (p. ej. una consulta) no debe
propagarse al trigger y descartar el lote.
        erroresInesperados++;
        System.debug(LoggingLevel.ERROR, 'SenialSensorHandler abortó el lote de '
+ seniales.size() + ' señales: ' + e.getTypeName());
    }
}
```

11.3 Clasificación por umbrales

Los límites salen de los registros de `Umbral__c`, no del código. Se prueban en el orden Inválida, Crítica y Advertencia, y gana el primer límite que se cumple; si ninguno se cumple, la señal es Normal. Sin umbral o con otra unidad, no se clasifica.

Archivo `classes/SenialSensorHandler.cls`, líneas 724 a 791 (con partes omitidas).

```
private static Clasificacion clasificar(Senial_Sensor__e senial, Map<String,
List<Umbral__c>> umbralesPorTipo) {
    Boolean esConectividad = senial.Message_Type__c == MENSAJE_CONECTIVIDAD;
    String medicion = esConectividad ? MEDICION_CONECTIVIDAD :
normalizar(senial.Measurement_Type__c);
    String valorTexto = esConectividad ? senial.Gap_Seconds__c : senial.Value__c;
    String unidad = esConectividad ? UNIDAD_CONECTIVIDAD : senial.Unit__c;
    // … arma el resultado con la clave activo|medición, el valor y la fecha de
origen
    if (resultado.valor == null) {
        return resultado.marcar('DatoInvalido', null, 'Valor ausente o no
numérico: ' + valorTexto);
    }
    if (resultado.occurredAt == null) {
        return resultado.marcar('DatoInvalido', null, 'occurredAt ausente o no
ISO 8601: ' + senial.Occurred_At__c);
    }
    String lectura = medicion + ' ' + resultado.valor + ' ' + unidad;
```

```
    List<Umbral__c> umbrales =
umbralesPorTipo.get(normalizar(senial.Asset_Type__c) + '|' + medicion);
    if (umbrales == null) {
        return resultado.marcar('SinUmbral', null, 'No hay umbral activo para ' +
senial.Asset_Type__c + ' ' + medicion);
    }
    List<Umbral__c> umbralesDeLaUnidad = new List<Umbral__c>();
    for (Umbral__c umbral : umbrales) {
        if (umbral.Unidad_Esperada__c == unidad) {
            umbralesDeLaUnidad.add(umbral);
        }
    }
    if (umbralesDeLaUnidad.isEmpty()) {
        return resultado.marcar('MedicionIncompatible', null,
            lectura + ' no está en la unidad esperada ' +
umbrales[0].Unidad_Esperada__c);
    }

    for (String severidad : SEVERIDADES_EN_ORDEN) {
        for (Umbral__c umbral : umbralesDeLaUnidad) {
            if (umbral.Severidad__c == severidad && cumple(resultado.valor,
umbral)) {
                return resultado.marcar(severidad,
umbral.Severidad_Nivel__c?.intValue(),
                    lectura + ' ' + VERBO_POR_OPERADOR.get(umbral.Operador__c) +
' el límite '
                    + LIMITE_POR_SEVERIDAD.get(severidad) + ' ' +
umbral.Operador__c + ' ' + umbral.Valor__c.stripTrailingZeros().toPlainString());
            }
        }
    }
    return resultado.marcar('Normal', 0, lectura + ' dentro de los límites');
}

private static Boolean cumple(Decimal valor, Umbral__c umbral) {
    switch on umbral.Operador__c {
        when 'LT' {
            return valor < umbral.Valor__c;
        }
        when 'LE' {
            return valor <= umbral.Valor__c;
        }
        when 'GT' {
            return valor > umbral.Valor__c;
        }
        when 'GE' {
            return valor >= umbral.Valor__c;
        }
```

```
            when else {
                return false;
            }
        }
    }
```

11.4 Desempate de señales con la misma fecha (D003)

Para un mismo activo y tipo de medición gana la señal con fecha de origen más reciente; con la misma fecha, la de mayor severidad, después la de `messageId` menor y, por último, la de `source` menor. El orden de llegada nunca decide.

Archivo `classes/SenialSensorHandler.cls`, líneas 886 a 906.

```
// D003: occurredAt, luego severidad, luego messageId; si aun así empatan (mismo messageId en otra fuente),
// la identidad completa (que incluye source) cierra el desempate. El orden de llegada nunca decide.
// compareTo y no <: en String el < de Apex depende del locale del usuario y no distingue mayúsculas.
public Boolean ganaA(Clasificacion otra) {
    if (occurredAt != otra.occurredAt) {
        if (occurredAt == null) {
            return false;
        }
        return otra.occurredAt == null || occurredAt > otra.occurredAt;
    }
    Integer miNivel = nivel == null ? 0 : nivel;
    Integer suNivel = otra.nivel == null ? 0 : otra.nivel;
    if (miNivel != suNivel) {
        return miNivel > suNivel;
    }
    Integer porMensaje = (messageId ?? '').compareTo(otra.messageId ?? '');
    if (porMensaje != 0) {
        return porMensaje < 0;
    }
    return (source ?? '').compareTo(otra.source ?? '') < 0;
}
```

11.5 Severidad del activo: el máximo de sus lecturas vigentes

La severidad del activo no se calcula en código: es un campo de resumen (roll-up) de tipo máximo sobre `Lectura_Vigente__c`, una por tipo de medición. Una temperatura normal no tapa una presión crítica del mismo activo.

Archivo `objects/Asset/fields/Severidad_Nivel__c.field-meta.xml`, líneas 1 a 10.

```
<?xml version="1.0" encoding="UTF-8"?>
<CustomField xmlns="http://soap.sforce.com/2006/04/metadata">
    <fullName>Severidad_Nivel__c</fullName>
    <label>Severidad (nivel)</label>
    <description>Highest severity among the asset's current readings (Discovery
agreement). Roll-Up MAX over Lectura_Vigente__c.</description>
    <type>Summary</type>
    <summarizedField>Lectura_Vigente__c.Severidad_Nivel__c</summarizedField>
    <summaryForeignKey>Lectura_Vigente__c.Asset__c</summaryForeignKey>
    <summaryOperation>max</summaryOperation>
</CustomField>
```

11.6 Recepción: publicar no es procesar

Cada mensaje con identidad válida deja primero su fila `Log_Senial__c` con resultado Publicada (una identidad repetida reutiliza la fila existente) y solo entonces se publica el evento, en trozos de hasta 200. El mensaje sin identidad válida queda Rechazada en el log y no se publica.

Archivo `classes/TelemetriaIngesta.cls`, líneas 561 a 647 (con partes omitidas).

```
/**
 * Writes one prepared page. Bulk-safe: one insert, one update at most, and one
EventBus.publish per 200 events,
 * whatever the page size.
 */
private static Resumen escribir(PaginaPreparada p) {
    // …
    // Step 1: log rows. Partial success: a duplicate identity (resend/replay)
leaves the existing row alone.
    List<Log_Senial__c> porInsertar = new List<Log_Senial__c>(filasCandidatas);
    porInsertar.addAll(rechazos);
    List<Boolean> conFila = new List<Boolean>(); // parallel to candidatos: may
the event be published?
    if (!porInsertar.isEmpty()) {
        List<Database.SaveResult> resultados = Database.insert(porInsertar,
false, AccessLevel.SYSTEM_MODE);
        // …
    }
    // …
    // Step 2: publish. Every event with a log row goes out, even a resend, so
the subscriber can count it.
    List<Log_Senial__c> aMarcarFallidas = new List<Log_Senial__c>();
    for (Integer desde = 0; desde < eventos.size(); desde += 200) {
        Integer hasta = Math.min(desde + 200, eventos.size());
        List<Senial_Sensor__e> trozo = new List<Senial_Sensor__e>();
        for (Integer i = desde; i < hasta; i++) {
            trozo.add(eventos[i]);
```

```
        }
        List<Database.SaveResult> resultados = EventBus.publish(trozo);
        // … cada resultado fallido suma a fallosPublicacion y, si la fila la
creó esta corrida, se marca para pasar a Fallida
    }

    // Step 3: a row left as 'Publicada' for an event that never went out would
lie, so mark it.
    // …
}
```

11.7 Programación de la ingesta cada 5 minutos

El cron de Salesforce no admite «cada 5 minutos» en una sola expresión, así que se agendan 12 jobs, uno por cada minuto 0, 5, …, 55. Este método los agenda y antes quita los que ya existían, para que queden siempre 12.

Archivo `classes/TelemetriaIngestaProgramada.cls`, líneas 77 a 96.

```
/**
 * Schedules 12 cron jobs (minute 0, 5, ..., 55) so ingestion runs every 5
minutes. Re-running first aborts the
 * existing jobs of this prefix, so calling it twice still leaves exactly 12.
Leaves the panel marker "on".
 * @param escenario simulator scenario for every run; blank means MIXED
 * @param paginas pages per run; null or less than 1 means 5
 * @return the number of jobs scheduled
 */
public static Integer programar(String escenario, Integer paginas) {
    desprogramar();
    String esc = String.isBlank(escenario) ? ESCENARIO_POR_DEFECTO : escenario;
    Integer p = (paginas == null || paginas < 1) ? PAGINAS_POR_DEFECTO : paginas;
    Integer total = 0;
    for (Integer minuto = 0; minuto < 60; minuto += INTERVALO_MINUTOS) {
        String cron = '0 ' + minuto + ' * * * ?';
        System.schedule(nombre(esc, minuto), cron, new
TelemetriaIngestaProgramada(esc, p));
        total++;
    }
    SesionSimulador.guardarProgramacion(esc);
    return total;
}
```

11.8 Un componente LWC: crear una intervención manual

El componente solo captura el motivo y llama al controlador Apex con el activo y el motivo; el permiso, las lecturas y que no haya otra intervención abierta se comprueban en el servidor.

Archivo `lwc/novaCasaCrearIntervencion/novaCasaCrearIntervencion.js`, líneas 1 a 73 (con partes omitidas).

```
import { api } from 'lwc';
import LightningModal from 'lightning/modal';
import crearIntervencion from
'@salesforce/apex/IntervencionOperadorController.crearIntervencion';
import { reduceError } from 'c/novaCasaEtiquetas';

// IntervencionOperadorController.MOTIVO_MAX
const MOTIVO_MAX = 1000;

// Manual intervention in the layout of a native "New record" dialog. Not the
native Case form: the operator has no Create
// on Case, and crearIntervencion checks the permission, the readings and that no
intervention is open, and sets the
// priority from the current severity (D020). The fields above the reason only
show what the server will do.
export default class NovaCasaCrearIntervencion extends LightningModal {
    @api assetId;
    @api assetName;
    // … propiedades públicas, getters y manejadores
    async handleSave() {
        const field = this.template.querySelector('lightning-textarea');
        if (field && !field.reportValidity()) {
            return;
        }
        if (this.saveDisabled) {
            return;
        }
        this.saving = true;
        this.disableClose = true;
        this.errorMessage = '';
        try {
            const result = await crearIntervencion({ assetId: this.assetId,
motivo: this.motivo });
            this.disableClose = false;
            this.close({ caseId: result.caseId, caseNumber: result.caseNumber });
        } catch (error) {
            this.errorMessage = reduceError(error);
        } finally {
            this.saving = false;
            this.disableClose = false;
        }
```

```
    }
}
```

12. Inventario completo de componentes

Lista de todo lo que el equipo diseñó y creó, tal como está en la confirmación `d87c049` de la rama `main`. Se armó a partir del árbol de archivos del repositorio, para que no falte ningún componente ni sobre ninguno: son 135 elementos en 22 grupos, más los 79 campos personalizados, que se describen objeto por objeto en las secciones 3.1 a 3.3. El código y la metadata de cada componente están en el repositorio `jpastorSFDC/nova-casa-telemetria`, dentro de `force-app/main/default/`; los scripts y los documentos, en sus propias carpetas. Este documento explica cómo funcionan y por qué; el repositorio tiene el código completo. La misma confirmación se entrega también como archivo zip del proyecto (`nova-casa-telemetria-d87c049.zip`), una copia de sus archivos sin el historial de git y sin la carpeta `docs/entrega-final`, porque estos documentos se entregan aparte.

Campos personalizados por objeto: `Log_Senial__c` 24, `Senial_Sensor__e` 17, `Lectura_Vigente__c` 12, `Case` 10, `Umbral__c` 9, `Asset` 6, `Account` 1. La descripción de cada uno está en la **sección 3**.

12.1 Apex: clases de la solución

Clase	Qué es	Dónde se explica y cómo se prueba
`ActivosOperadorController`	Controlador de lectura de la pantalla del operador: edificios, activos, historial, estado del activo y qué botones ve cada persona.	**sección 4** · prueba ActivosOperadorControllerTest
`CatalogoService`	Trae el catálogo del simulador y crea o actualiza edificios y activos; siembra los umbrales de referencia una sola vez.	**sección 4** · prueba CatalogoServiceTest
`IngestaController`	Controlador del botón «Traer señales»: valida el permiso y arranca la ingesta (D020).	**sección 4** · prueba IngestaControllerTest
`IntervencionOperadorController`	Controlador de escritura de la pantalla del operador: intervención manual, seguimiento y cierre en bloque, con el permiso verificado en el servidor.	**sección 4** · prueba IntervencionOperadorControllerTest
`IntervencionService`	Abre o enlaza la intervención (Case) de una señal crítica ganadora: una abierta de telemetría por activo (D039).	**sección 4** · prueba IntervencionServiceTest
`OpenCaseCheck`	Responde sí o no a si un activo ya tiene un Case abierto, en modo sistema, para no duplicar.	**sección 4** · prueba OpenCaseCheckTest
`SenialSensorHandler`	Manejador del trigger del evento: clasifica contra Umbral__c, decide reenvío, conflicto y atraso, elige la ganadora y pide la intervención.	**sección 4** · pruebas SenialSensorHandlerTest, LogAccionReintentoTest y PermisosProcesamientoTest

Clase	Qué es	Dónde se explica y cómo se prueba
`SesionSimulador`	Guarda el cursor y el resumen de la última corrida en Platform Cache, nunca en registros (D025).	sección 4 · prueba SesionSimuladorTest
`SimuladorClient`	Cliente HTTP del simulador por Named Credential: sesión y páginas de telemetría. No hace DML.	sección 4 · prueba SimuladorClientTest
`TelemetriaIngesta`	Queueable que pide páginas al simulador, deja la fila Publicada de cada mensaje con identidad válida y publica el evento.	sección 4 · prueba TelemetriaIngestaTest
`TelemetriaIngestaProgramada`	Schedulable que encola la ingesta; se agendan 12 jobs para cubrir «cada 5 minutos» (D040).	sección 4 · prueba TelemetriaIngestaProgramadaTest
`UmbralValidador`	Valida al guardar un Umbral__c que no queden rangos contradictorios.	sección 4 · pruebas UmbralValidadorTest y UmbralPermisosTest

12.2 Apex: triggers

Trigger	Qué es	Dónde se explica
`SenialSensorTrigger`	Sobre Senial_Sensor__e (after insert): entrega el lote de hasta 200 eventos a SenialSensorHandler.procesar.	sección 4 · fragmento 11.1
`UmbralTrigger`	Sobre Umbral__c (before insert, before update): llama a UmbralValidador.validar.	sección 4

12.3 Apex: clases de prueba

Clase de prueba	Qué es	Dónde se explica
`AccesoFueraDeJerarquiaTest`	Qué alcanza un usuario fuera de la jerarquía de roles del dueño de un edificio (BR-208, T8.5), comprobado sobre el resultado concreto.	sección 9
`ActivosOperadorControllerTest`	ActivosOperadorController: lo que devuelve la lectura de la pantalla del operador.	sección 9
`CaseCloseNeedsPermissionRuleTest`	Regla de validación Cierre_Exige_Permiso_Seguimiento (D052): cerrar una intervención exige el permiso Nova_Casa_Seguir_Intervencion.	sección 9
`CaseStatusCommentRuleTest`	Regla de validación Comentario_Obligatorio_Cambio_Estado (D046): cambiar el estado exige cambiar la descripción en la misma edición.	sección 9
`CatalogoServiceTest`	CatalogoService con GET /catalog simulado; nunca llama al simulador real.	sección 9
`IngestaControllerTest`	IngestaController (D020): el botón encola la ingesta y no lanza otra mientras hay una en curso.	sección 9

Clase de prueba	Qué es	Dónde se explica
`IntervencionOperadorControllerTest`	IntervencionOperadorController (D020, D036) y las capacidades de ActivosOperadorController, con los permission sets reales: falla si a uno se le quita un permiso personalizado.	sección 9
`IntervencionServiceTest`	IntervencionService: una intervención de telemetría abierta por activo, refrescada con su crítica más reciente (D039).	sección 9
`LogAccionReintentoTest`	Ata la fórmula Accion_Reintento__c de Log_Senial__c a lo que el manejador hace de verdad (BR-209, D014).	sección 9
`OpenCaseCheckTest`	OpenCaseCheck responde lo mismo sin importar quién lo ejecute, aunque el usuario no vea el Case abierto.	sección 9
`PermisosProcesamientoTest`	Protege Nova_Casa_Procesamiento, el del usuario de integración: falla si gana eliminar, Modify All, ModifyAllData o ViewAllData, o si pierde crear sobre Lectura_Vigente__c, Log_Senial__c, Case o Senial_Sensor__e; y comprueba que un usuario sin permisos de operador ni de administrador procesa una señal normal y una crítica (BR-208, D017). No prueba que ese conjunto sea lo que habilita el procesamiento, que corre en modo sistema.	sección 9
`SeguridadTest`	Matriz de seguridad (BR-208, BR-210, T8.5): un usuario por cada permission set de persona, sobre el perfil Minimum Access, con System.runAs.	sección 9
`SenialSensorHandlerTest`	Suite del manejador: clasificación, lotes, duplicados y conflictos, señales atrasadas, desempate, severidad e intervención.	sección 9
`SesionSimuladorTest`	SesionSimulador (HP-05, D025): guardar y leer el cursor; la caché real no existe en pruebas, así que usa un mapa de prueba.	sección 9
`SimuladorClientTest`	SimuladorClient con HttpCalloutMock y cursores falsos; nunca llama al simulador real.	sección 9
`TelemetriaIngestaProgramadaTest`	La programación encola la ingesta (US-201, BR-201, D040); el Queueable no se ejecuta.	sección 9
`TelemetriaIngestaTest`	TelemetriaIngesta con HttpCalloutMock: páginas, filas del log, publicación y cursor. Nunca llama al simulador real.	sección 9
`UmbralPermisosTest`	BR-204 y BR-210: los permission sets reales deciden quién edita los umbrales.	sección 9
`UmbralValidadorTest`	UmbralValidador y UmbralTrigger (US-204, D018): una Advertencia no puede quedar inalcanzable frente a la Crítica de su combinación.	sección 9

12.4 Componentes LWC

Componente	Qué es	Dónde se explica
`novaCasaActivoEstado`	Estado y detalle de un activo.	sección 6

Componente	Qué es	Dónde se explica
`novaCasaActivosOperador`	Monitor principal del operador: lista de activos, filtros, antigüedad y estados de carga, vacío y error.	sección 6
`novaCasaCrearIntervencion`	Ventana «Nueva intervención» para abrir una intervención manual (HP-07, D045).	sección 6 · fragmento 11.8
`novaCasaEdificioResumen`	Tarjeta de resumen del edificio, con conteos por severidad.	sección 6
`novaCasaEtiquetas`	Módulo de utilidades compartido, sin HTML propio: etiquetas de tipo de medición, unidad y tipo de señal, severidad (severityUi), antigüedad relativa (relativeAge), texto de apertura de la intervención (openingText) y reduceError.	sección 6
`novaCasaHistorialSenales`	Historial de señales de un activo; desde D048 no está montado en ninguna página.	sección 6
`novaCasaIlustracion`	Ilustración SVG por tipo de activo o de edificio, teñida según la severidad (HP-07, D042).	sección 6
`novaCasaIntervencion`	Panel de la página de registro de la intervención (Case), de solo lectura.	sección 6

12.5 Páginas Lightning

Página	Qué es	Dónde se explica
`Activos_Operador`	Página de aplicación «Activos del operador»: aloja novaCasaActivosOperador.	sección 6
`Nova_Casa_Activo_Record_Page`	Página de registro de Asset («Nova Casa Activo»): novaCasaActivoEstado, panel de aspectos destacados, detalles y listas relacionadas.	sección 6
`Nova_Casa_Edificio_Record_Page`	Página de registro de Account («Nova Casa Edificio»): novaCasaEdificioResumen, panel de aspectos destacados, detalles y listas relacionadas.	sección 6
`Nova_Casa_Intervencion_Record_Page`	Página de registro de Case («Nova Casa Intervención»): novaCasaIntervencion, el Path nativo, detalles y listas relacionadas.	sección 6
`Nova_Casa_Lectura_Record_Page`	Página de registro de Lectura_Vigente__c: panel de aspectos destacados, detalles y listas relacionadas, sin componente propio.	sección 3.2
`Nova_Casa_Senial_Record_Page`	Página de registro de Log_Senial__c: panel de aspectos destacados, detalles y listas relacionadas, sin componente propio.	sección 3.2
`Nova_Casa_Umbral_Record_Page`	Página de registro de Umbral__c: panel de aspectos destacados, detalles y listas relacionadas, sin componente propio.	sección 3.2

12.6 Aplicación y pestañas

Elemento	Qué es	Dónde se explica
`Activos_Operador`	Pestaña «Activos del operador», que abre la página de aplicación del mismo nombre.	sección 6

Elemento	Qué es	Dónde se explica
`Log_Senial__c`	Pestaña del objeto Log_Senial__c.	sección 3.2
`Nova_Casa`	Aplicación «Nova Casa Telemetry»: navegación con Activos del operador, Cuentas (edificios), Activos, Casos, Umbrales, Registros de Señal, Informes y Paneles (las dos últimas solo las ve quien puede ejecutar informes, D060).	sección 6
`Umbral__c`	Pestaña del objeto Umbral__c.	sección 3.2

12.7 Page layouts

Layout	Qué es	Dónde se explica
`Account-Account Layout`	Edificio: secciones Edificio, Ubicación e Información del sistema; listas relacionadas de activos y casos.	sección 3.1
`Asset-Asset Layout`	Activo: secciones Activo, Estado actual e Información del sistema; lecturas vigentes, casos y señales relacionados.	sección 3.1
`Case-Case Layout`	Intervención: secciones Intervención, Activo, Descripción, Contacto (opcional), Origen de la señal y Última señal crítica; señales enlazadas.	sección 3.1
`Lectura_Vigente__c-Lectura Vigente Layout`	Lectura vigente: secciones Lectura vigente, Antigüedad, Técnico e Información del sistema.	sección 3.2
`Log_Senial__c-Registro de Señal Layout`	Registro de Señal: secciones Resultado, Tiempos, Reenvíos, conflictos y reintento, Identidad e Información del sistema.	sección 3.2
`Umbral__c-Umbral Layout`	Umbral: secciones Umbral, Regla e Información del sistema.	sección 3.2

12.8 Compact layouts

Compact layout	Qué es	Dónde se explica
`Account.Nova_Casa_Edificio`	Diseño compacto del edificio (encabezado del registro).	sección 3.1
`Asset.Nova_Casa_Asset`	Diseño compacto del activo.	sección 3.1
`Case.Nova_Casa_Intervencion`	Diseño compacto de la intervención.	sección 3.1
`Lectura_Vigente__c.Nova_Casa_Lectura`	Diseño compacto de la lectura vigente.	sección 3.2
`Log_Senial__c.Nova_Casa_Senial`	Diseño compacto del registro de señal.	sección 3.2
`Umbral__c.Nova_Casa_Umbral`	Diseño compacto del umbral.	sección 3.2

12.9 Vistas de lista

Vista de lista	Qué es	Dónde se explica
`Account.Nova_Casa_Edificios`	«Nova Casa - Edificios»: nombre, ciudad e identificador del edificio.	sección 3.1

Vista de lista	Qué es	Dónde se explica
`Asset.Nova_Casa_Activos`	«Nova Casa - Activos»: activo, edificio, tipo, severidad actual, última señal y minutos sin señal.	sección 3.1
`Case.Nova_Casa_Intervenciones_Abiertas`	«Intervenciones abiertas»: casos no cerrados con origen Telemetria u Operador.	sección 3.1
`Log_Senial__c.Atrasadas`	«Atrasadas»: filas con resultado Atrasada.	sección 3.2
`Log_Senial__c.Conflictos`	«Conflictos»: filas con resultado Conflicto o con al menos un conflicto contado (una fila ya procesada conserva su resultado cuando llega un conflicto, D047).	sección 3.2
`Log_Senial__c.Duplicadas`	«Duplicadas (reenvíos)»: filas con resultado Duplicada o con al menos un reenvío.	sección 3.2
`Log_Senial__c.Fallidas`	«Fallidas»: filas con resultado Fallida.	sección 3.2
`Log_Senial__c.Rechazadas`	«Rechazadas»: filas con resultado Rechazada.	sección 3.2
`Log_Senial__c.Todas_las_senales`	«Todas las señales»: todas las filas del log, sin filtro.	sección 3.2
`Umbral__c.Nova_Casa_Umbrales`	«Nova Casa - Umbrales»: descripción, tipo de activo, tipo de medición, severidad y si está activo.	sección 3.2

12.10 Path

Path	Qué es	Dónde se explica
`Cases_Path`	Path nativo del Case (D044) con los pasos New, En curso, On Hold, Escalated y Closed.	sección 3.1

12.11 Objetos

Objeto	Qué es	Dónde se explica
`Account`	Estándar. Edificio; propiedad de un operador y OWD Privado (D005, D022).	Modelo de Datos, sección 2
`Asset`	Estándar. Activo con sensor; Controlado por el padre (Account).	Modelo de Datos, sección 2
`Case`	Estándar. Intervención operativa: una abierta de telemetría por activo (US-205, D039).	Modelo de Datos, sección 2
`Contact`	Estándar. El modelo no lo usa: se versiona su acceso Controlado por el padre dentro del cambio de OWD de T8.2 (docs/acceso.md); los permission sets le dan Read porque dar Read sobre Account exige Read Contact (D017).	Modelo de Datos, sección 2

Objeto	Qué es	Dónde se explica
`Lectura_Vigente__c`	Personalizado. La última lectura válida por activo y tipo de medición (BR-203).	sección 3.2
`Log_Senial__c`	Personalizado. Bitácora: una fila por señal recibida, con su resultado de ingesta (BR-209).	sección 3.2
`Opportunity`	Estándar. El modelo no lo usa: el archivo solo declara el acceso Privado (interno y externo), para que los roles se puedan desplegar (D058).	sección 3.1
`Senial_Sensor__e`	Platform Event de alto volumen: la señal del simulador tal como llega (BR-201).	sección 3.3
`Umbral__c`	Personalizado. Límites de clasificación por tipo de activo y de medición, editables sin desplegar (BR-204).	sección 3.2

12.12 Reglas de validación

Regla	Qué es	Dónde se explica
`Case.Cierre_Exige_Permiso_Seguimiento`	Cerrar una intervención exige el permiso Nova_Casa_Seguir_Intervencion (D052).	sección 8
`Case.Comentario_Obligatorio_Cambio_Estado`	Cambiar el estado de un Case exige cambiar la descripción en la misma edición (D046).	sección 8

12.13 Valores de picklist estándar

Conjunto de valores	Qué es	Dónde se explica
`CaseOrigin`	Valores del origen del Case: agrega Telemetria y Operador.	sección 3.1
`CasePriority`	Valores de prioridad del Case: agrega Critica.	sección 3.1
`CaseStatus`	Valores de estado del Case: agrega En curso.	sección 3.1

12.14 Configuración y recursos

Elemento	Qué es	Dónde se explica
`settings/Case`	Configuración de Case: se puede cerrar un caso cambiando su estado.	sección 3.1
`staticresources/NovaCasaUmbralesSemilla`	Recurso estático con los valores de referencia con que CatalogoService siembra Umbral__c (D054).	sección 5

12.15 Permission sets

Permission set	Qué es	Dónde se explica
`Nova_Casa_Admin`	Administrador de la telemetría: crear, leer, editar y eliminar, con View All y Modify All, sobre Account, Asset, Case, Contact, Lectura_Vigente__c, Log_Senial__c y Umbral__c; acceso a los campos del modelo, a los tres permisos personalizados (incluido «Traer señales») y a las pestañas de Registros de Señal y Umbrales (HP-03, T2.9, D020); no administra usuarios ni la seguridad de la org (D050).	Permisos de Acceso, sección 4
`Nova_Casa_Coordinator`	Todo lo del operador, más editar el Case para el seguimiento de intervenciones y leer, crear y editar Umbral__c con su pestaña (BR-204, D004, D020).	Permisos de Acceso, sección 4
`Nova_Casa_Gerente`	Gerente regional de solo lectura (BR-208, D022, docs/acceso.md); ejecuta informes sin exportarlos y ve las pestañas de Informes y Paneles (D060).	Permisos de Acceso, sección 4
`Nova_Casa_Lightning`	Solo acceso a Lightning Experience, para que las personas de Nova Casa usen el perfil Minimum Access.	Permisos de Acceso, sección 4
`Nova_Casa_Operator`	Camino del operador en la pantalla de telemetría (BR-208, D005, D020).	Permisos de Acceso, sección 4
`Nova_Casa_Procesamiento`	Usuario de integración sin interfaz (BR-208, D017): lee todos los edificios y activos (View All), crea y edita lecturas vigentes, casos y filas del log, crea y lee el evento Senial_Sensor__e y solo lee los umbrales.	Permisos de Acceso, sección 4
`Nova_Casa_Simulator_Integration`	Permite al usuario de integración autenticarse como el principal del simulador (credencial externa).	sección 8

12.16 Permisos personalizados

Permiso	Qué es	Dónde se explica
`Nova_Casa_Crear_Intervencion`	Abrir una intervención manual desde la pantalla del operador (D020).	sección 8 · Permisos de Acceso, sección 7
`Nova_Casa_Seguir_Intervencion`	Cambiar el estado de una intervención abierta desde la pantalla del operador (D020).	sección 8 · Permisos de Acceso, sección 7
`Nova_Casa_Traer_Senales`	Arrancar una corrida de ingesta desde la pantalla del operador (D020).	sección 8 · Permisos de Acceso, sección 7

12.17 Roles

Rol	Qué es	Dónde se explica
`Coordinador_Mantenimiento`	Coordinador; debajo del gerente.	Permisos de Acceso, sección 3
`Gerente_Regional_Operaciones`	Rol más alto de la jerarquía de Nova Casa: gerente regional.	Permisos de Acceso, sección 3

Rol	Qué es	Dónde se explica
`Integracion_Telemetria`	Usuario de integración; debajo del coordinador, para que este y el gerente vean los Case que crea (D022).	Permisos de Acceso, sección 3
`Operador_Edificios`	Operador; debajo del coordinador. Cada edificio es de un operador.	Permisos de Acceso, sección 3

12.18 Perfil

Perfil	Qué es	Dónde se explica
`Minimum Access - Salesforce`	Perfil de las personas de Nova Casa y del usuario de integración; se versionó con cinco asignaciones de page layout (Asset, Case, Lectura_Vigente__c, Log_Senial__c y Umbral__c) y cinco permisos de usuario estándar (D026, D017).	Permisos de Acceso, sección 4

12.19 Integración y plataforma

Elemento	Qué es	Dónde se explica
`cachePartitions/NovaCasaSimulador`	Partición de Platform Cache con el cursor de la sesión y el último resumen de la ingesta (D025).	sección 4
`externalCredentials/Nova_Casa_Simulator`	External Credential: define el principal NovaCasaSimulatorPrincipal; el token Bearer (parámetro BearerToken) se carga solo en Setup y nunca está en el repositorio (CONTRIBUTING.md); el nombre se conservó por D002.	sección 8
`namedCredentials/Nova_Casa_Simulator`	Named Credential: guarda la dirección base del simulador y arma el encabezado Authorization con el BearerToken de la External Credential; el nombre se conservó por D002.	sección 8
`platformEventSubscriberConfigs/Nova_Casa_Procesamiento`	Configuración del suscriptor del evento: corre como el usuario de integración (D017).	sección 7

12.20 Scripts y configuración del proyecto

Archivo	Qué es	Dónde se explica
`jest-mocks/lightning/modal.js`	Reemplazo de lightning/modal para las pruebas Jest (#113).	sección 9
`jest.config.js`	Configuración de Jest.	sección 9
`package.json`	Dependencias y scripts de las pruebas Jest.	sección 9
`scripts/apex/desprogramar-ingesta.apex`	Quita los jobs de la ingesta, antes de limpiar o desplegar.	sección 7

Archivo	Qué es	Dónde se explica
`scripts/apex/programar-ingesta.apex`	Agenda los 12 jobs de la ingesta; se ejecuta en cada org nueva (D040).	sección 7
`scripts/org-check.sh`	Comprobaciones contra la org: consultas, logs, pruebas, desfase con main, limpieza y corridas de escenarios.	sección 9
`scripts/org-drift-compare.py`	Compara elemento por elemento la metadata de la org con la de main, para el desfase de org-check.sh.	sección 9
`sfdx-project.json`	Proyecto Salesforce DX (API 67.0); su entrada replacements cambia al desplegar el marcador del panel del gerente por el valor de la variable NOVA_CASA_GERENTE_USERNAME (D060).	-

12.21 Documentación del repositorio

Archivo	Qué es	Dónde se explica
`CONTRIBUTING.md`	Ramas, commits, pull requests y reglas de despliegue.	sección 9
`docs/acceso.md`	Quién ve y hace qué: roles, permission sets, personas y verificaciones.	Permisos de Acceso
`docs/br-201-210.md`	Reglas de negocio BR-201 a BR-210, textuales del MDSS.	Documentación Funcional
`docs/decisiones.md`	Decisiones D001 a D060, cada una con su alternativa descartada y su trade-off.	sección 10
`docs/entrega.md`	Estado por historia, evidencia, corridas de escenarios y contrato del mensaje.	sección 9
`docs/muestras/catalog.json`	El catálogo real del simulador.	sección 3.3
`docs/muestras/telemetry.json`	Una página real de 200 mensajes del simulador.	sección 3.3
`README.md`	Guía del repositorio.	-

12.22 Panel e informes del gerente

Elemento	Qué es	Dónde se explica
`dashboards/Nova_Casa_Gerencia`	Carpeta de paneles «Nova Casa Gerencia», compartida solo con el rol Gerente_Regional_Operaciones, con permiso de ver (D060).	Documentación Funcional, sección 9.1
`dashboards/Nova_Casa_Gerencia_Regional`	Panel «Nova Casa: Gerencia regional»: cinco componentes sobre cinco informes, estático y ejecutado como la persona gerente; guarda un marcador en lugar de su nombre de usuario (D060).	Documentación Funcional, sección 9.1 · CONTRIBUTING.md
`reports/Nova_Casa_Activos_Por_Severidad`	Informe «Estado de todos los activos»: cuántos activos de todos los edificios están en cada estado.	Documentación Funcional, sección 9.1

Elemento	Qué es	Dónde se explica
`reports/Nova_Casa_Edificios_A_Atender`	Informe matricial «Activos de cada edificio por estado»: una fila por edificio y una columna por estado.	Documentación Funcional, sección 9.1
`reports/Nova_Casa_Gerencia`	Carpeta de informes «Nova Casa Gerencia», compartida solo con el rol Gerente_Regional_Operaciones, con permiso de ver (D060).	Documentación Funcional, sección 9.1
`reports/Nova_Casa_Intervenciones_Abiertas`	Informe «Intervenciones abiertas por edificio»: dónde están las intervenciones abiertas, por edificio y tipo de activo.	Documentación Funcional, sección 9.1
`reports/Nova_Casa_Intervenciones_Antiguas`	Informe «Intervenciones abiertas más antiguas»: las 10 que más tiempo llevan esperando.	Documentación Funcional, sección 9.1
`reports/Nova_Casa_Telemetria_Por_Dia`	Informe «Intervenciones de los últimos 30 días»: cuántas se crearon en cada edificio, automáticas y manuales.	Documentación Funcional, sección 9.1
`reportTypes/Nova_Casa_Activos`	Tipo de informe «Nova Casa: activos con edificio», sobre Asset con su cuenta, sin Producto ni Contacto: el gerente no lee Product2 (D060).	Documentación Funcional, sección 9.1

13. Puesta en marcha en una org nueva

Orden para dejar la solución funcionando en una org nueva, por ejemplo la que reemplace a la org de prueba, un trial que vence el 19 de noviembre de 2026 (D040). Resume `README.md`, `docs/acceso.md` y `CONTRIBUTING.md`; allí está el detalle. Hace falta la Salesforce CLI (`sf`).

1. **Clonar y conectar.**

```
git clone https://github.com/jpastorSFDC/nova-casa-telemetria.git
sf org login web -a nova-cdo --instance-url https://<dominio-de-la-
org>.my.salesforce.com
sf config set target-org nova-cdo
```

2. **Desplegar desde `main`, validando primero.** El despliegue completo de `force-app` se hace solo desde `main` (`CONTRIBUTING.md`). En una org nueva se deja fuera la carpeta `platformEventSubscriberConfigs`, porque su campo `user` es el nombre de usuario del integrador y solo se puede desplegar cuando ese usuario existe (D017); se despliega justo después del paso 3. El panel del gerente (D060) corre como esa persona y su archivo guarda un marcador en lugar de su nombre de usuario: `sfdx-project.json` lo cambia al desplegar por la variable de entorno `NOVA_CASA_GERENTE_USERNAME`, y sin ella el comando se detiene antes de llegar a la org. El comando de `CONTRIBUTING.md` («The manager dashboard and its running user») lee ese nombre de la org por el rol y falla a propósito si no hay exactamente un gerente activo; por eso, en una org nueva, la carpeta `dashboards` también se deja fuera aquí y se despliega después del paso 3 (este orden no se probó en una org nueva). Si la org es de producción, un despliegue con Apex exige pruebas con 75 %

de cobertura por clase y se hace con `--test-level RunSpecifiedTests`, listando las clases de prueba en `--tests`, nunca con `RunLocalTests` (`CONTRIBUTING.md`). Si el despliegue rechaza los roles por el nivel de acceso a Case u Opportunity frente al acceso general de la org, se despliegan primero los objetos y después los roles (`docs/acceso.md`). La partición de Platform Cache falla si la org no tiene capacidad asignada (D025).

```
sf project deploy start --dry-run --test-level RunSpecifiedTests --tests <clases
de prueba> \
  --source-dir force-app/main/default/classes --source-dir force-
app/main/default/objects \
  --source-dir <el resto de las carpetas de force-app/main/default, menos
platformEventSubscriberConfigs y dashboards>
# después, el mismo comando sin --dry-run
```

3. **Crear las personas y el usuario de integración.** Los usuarios son datos de la org, no metadata (D017). Cada persona va en el perfil `Minimum Access - Salesforce`, con su rol (Operador de Edificios, Coordinador de Mantenimiento o Gerente Regional de Operaciones; la persona Admin no lleva rol), su permission set de negocio y `Nova_Casa_Lightning`: coordinador y gerente ven los edificios solo por la jerarquía de roles (D022). El usuario de integración va en el mismo perfil, con el rol Integración Telemetría y los permission sets `Nova_Casa_Simulator_Integration` y `Nova_Casa_Procesamiento` (D017). El detalle está en **Permisos de Acceso**, secciones 3 y 4. Justo después, antes de la primera ingesta (paso 7), se despliega `platformEventSubscriberConfigs/Nova_Casa_Procesamiento`, cuyo campo `user` debe apuntar al nombre de usuario del integrador de esa org (D017). Sin esa configuración el trigger procesa como Automated Process, no como el usuario de integración, y un trigger que ya estaba activo no la toma hasta que se suspende y se reanuda.

```
sf org assign permset -o nova-cdo -n Nova_Casa_Operator -n Nova_Casa_Lightning -b
<usuario-del-operador>
sf project deploy start --source-dir force-
app/main/default/platformEventSubscriberConfigs
# con NOVA_CASA_GERENTE_USERNAME definida, el panel del gerente:
sf project deploy start --source-dir force-app/main/default/dashboards
```

4. **Cargar el token del simulador.** Un administrador lo ingresa en Setup, en la External Credential `Nova_Casa_Simulator` (principal `NovaCasaSimulatorPrincipal`, parámetro `BearerToken`). Nunca va en el repositorio ni en un archivo.

5. **Cargar el catálogo.** Se corre en Execute Anonymous con un usuario que tenga `Nova_Casa_Simulator_Integration` (autentica la llamada al simulador) y permisos para crear y editar Account y Asset y para leer y crear `Umbral__c`; si no los tiene, la carga falla. Crea o actualiza un edificio y un activo por cada uno del catálogo, sin duplicar, y siembra los umbrales de referencia una sola vez por combinación (**sección 5**). Se puede repetir sin riesgo.

```
CatalogoService.loadCatalog();   // en Execute Anonymous
```

6. **Asignar el dueño de cada edificio.** El catálogo no toca el dueño: un edificio nuevo es de quien corrió la carga y no lo ve ningún operador hasta que se reasigna a uno (D022). El administrador lo reasigna con la consulta de `docs/acceso.md`, sección «Building owners».
7. **Agendar la ingesta automática.** Crea los 12 jobs que cubren «cada 5 minutos» (D040). Se ejecuta con un usuario que tenga `Nova_Casa_Simulator_Integration` y Create sobre `Senial_Sensor__e`, porque los jobs corren como quien los agenda. Repetirlo deja siempre 12. Para quitarlos, antes de limpiar la org o de desplegar las clases agendadas, `desprogramar-ingesta.apex`.

   ```
   sf apex run --file scripts/apex/programar-ingesta.apex -o nova-cdo
   ```

8. **Comprobar.** Correr las pruebas con `scripts/org-check.sh tests`, abrir la vista «Todas las señales» de `Log_Senial__c` y, si se quiere ver el recorrido completo ya, usar el botón «Traer señales» de la pantalla del operador con un usuario que tenga `Nova_Casa_Traer_Senales` y `Nova_Casa_Simulator_Integration` (la persona Admin no tiene el segundo). Cada mensaje aparece como Publicada y pasa a su resultado cuando el suscriptor lo procesa.